



Build an AI Chatbot with Amazon Bedrock



yakubu abdullahi yusuf

ic is an **S3 bucket**. Here's what it can do:

boxes and belongings, an S3 bucket stores all your files and data in the cloud. You can store anything: photos, videos, documents, ba
ent boxes for different things—one for old photos, one for winter clothes, etc. In an S3 bucket, you can create "folders" to organize
ur attic, you can get it anytime you want. Similarly, an S3 bucket lets you access your files from anywhere in the world, as long as
out unwanted visitors, you can set permissions on your S3 bucket to control who can see or access the data.

c as being super sturdy and never breaking down. S3 buckets are designed to be highly durable and available, meaning your data is sa

cal attic can just expand without any hassle. S3 buckets can easily scale to store as much data as you need.
Don't use. It's like paying only for the space you actually use in your attic.

nd scalable storage solution in the cloud, much like a magical, expandable, and secure attic in the sky!

a, a place where every book ever written is stored. Here's what it's like:

holds every conceivable book, an S3 bucket can store a vast amount of data. It could be photos, videos, documents, or any type of fi
rent sections for different types of books—history, science, fiction, etc. In an S3 bucket, you can create folders or "buckets" to o
nd it no matter where you are in the world. Similarly, you can access your files stored in an S3 bucket from anywhere with an interne
o make sure only certain people can read certain books. In an S3 bucket, you can set permissions to control who can access or modify
was renowned for its durability and accessibility. An S3 bucket is designed to be extremely reliable, ensuring your data is safe and

s, it can just expand. An S3 bucket can scale to accommodate your growing data needs.
it like how you might only borrow books you actually read, rather than buying every single one in the library.

ry of Alexandria, where you can store, organize, and access any amount of data securely and reliably.



Introducing Today's Project

In this project, I'm going to build a Python chatbot in AWS CloudShell that uses Amazon Bedrock's Converse API to chat with the Amazon Nova 2 Lite foundation model, maintaining conversation history across multiple messages. I want to learn this because it will strengthen my understanding of generative AI application development, cloud-based AI services, API integration, and conversational memory management.

Key tools and concepts

The key tools I used include AWS Bedrock (boto3 bedrock-runtime client), the Amazon Nova Lite foundation model, Python for scripting the chatbot logic, and a command-line interface (CloudShell) to run and test the application. Key concepts I learnt include How to build and interact with large language models using AWS Bedrock, prompt engineering through system prompts, controlling model behavior using inference parameters like temperature and top-p, and implementing safety guardrails to prevent harmful outputs. I also learned how conversational memory works in LLM applications and how system design choices affect response quality and alignment.

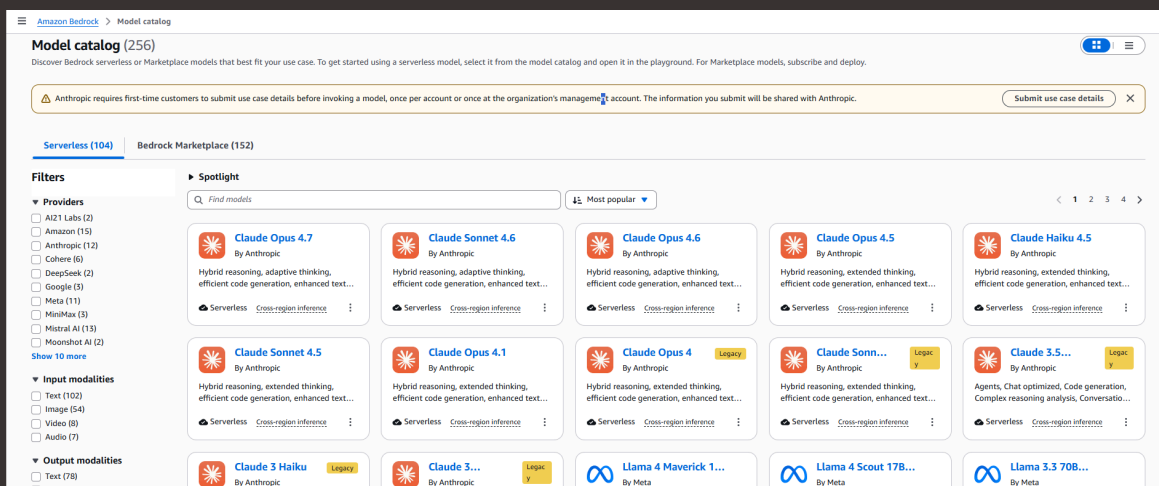
Challenges and wins

This project took me approximately 60 minutes. It was an interesting and beneficial project to do.



Discovering Amazon Bedrock

In this step, I'm going to log in to my AWS account, navigate to Amazon Bedrock in the console, explore and try out the various foundational models in the model catalog.



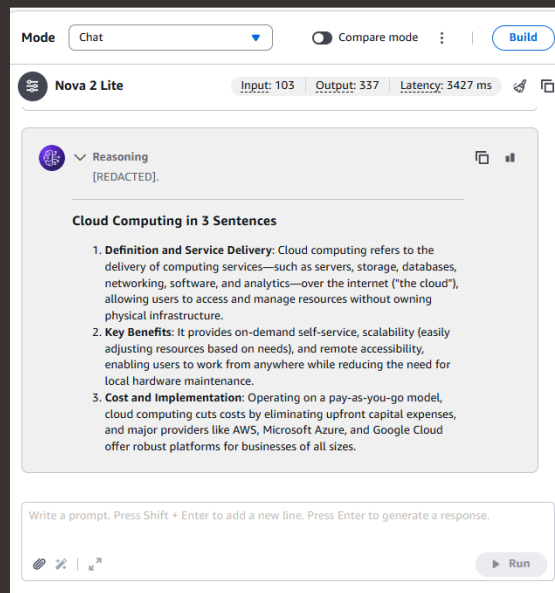
Understanding Foundation Models

I learned that a foundation model is a large model that has been pre-trained on massive datasets. Amazon Bedrock provides access to foundation models such as Amazon, Anthropic, Meta, Cohere, and others through a single API for users to use in developing their own models.



Chatting with an AI Model

I sent a prompt about cloud computing and the model responded by defining cloud computing, outlining cloud computing key benefits and the cost and implementation of cloud computing.





Writing My First AI API Call

In this step, I'm going to use CloudShell to write a python script that sends a question to Amazon Nova 2 lite and prints the answer using Bedrock converse API calls.

```
GNU nano 8.3                                     bedrock_chat.py
import boto3

# 1. Connect to Amazon Bedrock
client = boto3.client("bedrock-runtime", region_name="us-east-1")

# 2. Choose which AI model to use
model_id = "amazon.nova-lite-v1:0"

# 3. Create your prompt
messages = [
    {
        "role": "user",
        "content": [{"text": "What is cloud computing? Explain in 2 sentences."}]
    }
]

# 4. Send the message and get a response
response = client.converse(modelId=model_id, messages=messages)

# 5. Extract the text and print it
response_text = response["output"]["message"]["content"][0]["text"]
print(response_text)
```

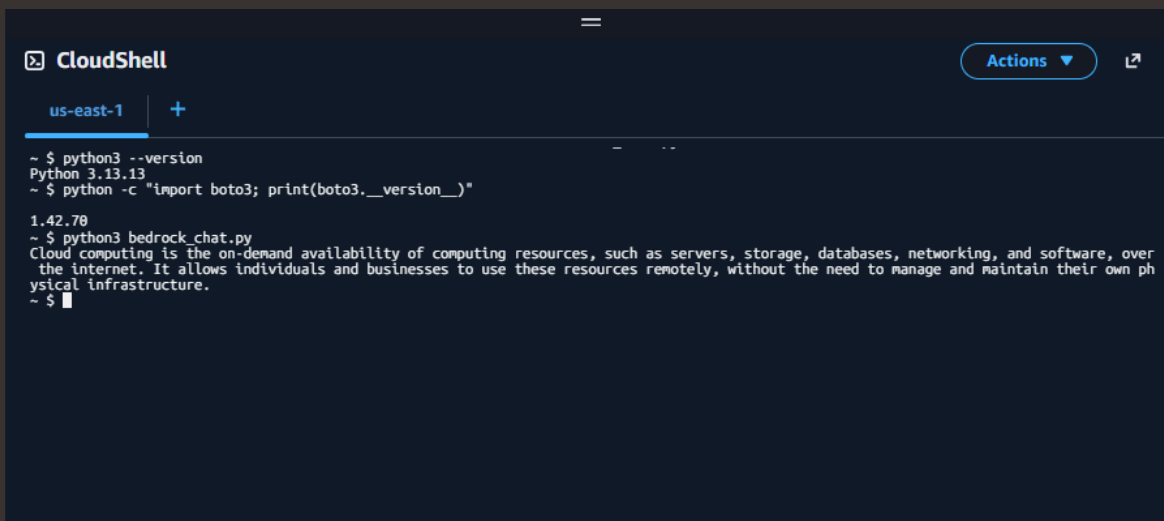
Understanding the Converse API

I created a client using boto3, The messages list contains the user prompts.



Running the Bedrock Script

I ran my script and the AI responded with the definition of cloud computing. The response came from the Amazon Nova Lite foundation model through Amazon Bedrock's Converse API.



```
CloudShell
us-east-1 +
~ $ python3 --version
Python 3.13.13
~ $ python -c "import boto3; print(boto3.__version__)"
1.42.70
~ $ python3 bedrock_chat.py
Cloud computing is the on-demand availability of computing resources, such as servers, storage, databases, networking, and software, over the internet. It allows individuals and businesses to use these resources remotely, without the need to manage and maintain their own physical infrastructure.
~ $
```



Building a Multi-Turn Chatbot

In this step, I am building my chatbot with a system prompt to give it a little bit of personality, then I will add a conversation loop with the history been saved and I will tune the chatbot's responses with inference parameters. Conversation history matters because it allows the chatbot to remember previous messages and maintain context throughout the interaction.

System Prompt and Inference Parameters

I configured the system prompt to give the chatbot a friendly teaching personality, instructing it to act as a cloud computing tutor that explains concepts in simple terms and uses analogies to improve understanding. I adjusted the inferenceConfig to control the behavior of the model's responses. I set the temperature to 0.7 to balance creativity and consistency, allowing the chatbot to generate natural but still relevant explanations. I also set topP to 0.9 to filter token selection and improve response quality, and limited maxTokens to 512 to control response length and prevent overly long outputs.

```
... $ python3 bedrock_chat_example.py
Chatbot ready! Type "quit" to exit.
You: What is an S3 bucket?
Bot: Sure thing! Let's imagine you have a huge, magical closet that can store anything you want, from clothes to books to toys. This closet is so special that it can grow as big as you need, and you can access it from anywhere in the world, whether you're at home, at work, or even on vacation.

Now, in the world of cloud computing, an S3 bucket is a bit like that magical closet, but instead of storing your clothes and toys, it stores data. Here's a breakdown of what an S3 bucket is and how it works:

### What is an S3 bucket?
**S3** stands for **Simple Storage Service**. It's a service offered by Amazon Web Services (AWS) that allows you to store and retrieve any amount of data at any time.

### Key Features:
1. **Scalability**: Just like your magical closet can grow as big as you need, an S3 bucket can store as much data as you want. You don't have to worry about running out of space.
2. **Accessibility**: You can access your data from anywhere in the world, just like you could check out your magical closet from any location.
3. **Durability**: Your data is stored safely and securely. AWS ensures that your data is protected and can be retrieved even if something goes wrong.
4. **Organization**: Inside your S3 bucket, you can organize your data into folders and files, just like you might organize your closet with shelves and bins.

### Analogy:
Imagine you have a photo album (your S3 bucket). Each photo in the album represents a file, and each album can have many pages (folders). You can add as many photos (files) as you want, and you can create new albums (folders) whenever you need to. Plus, you can share your photo album with friends and family (access your data from anywhere), and you can trust that your photos will be safe and sound, even if the album gets misplaced (durability).

### Use Cases:
- **Backup and Restore**: Store important files and documents so you can easily recover them if needed.
- **Media Storage**: Save photos, videos, and music files for easy access and sharing.
- **Data Lakes**: Store large amounts of raw data for analysis and processing.

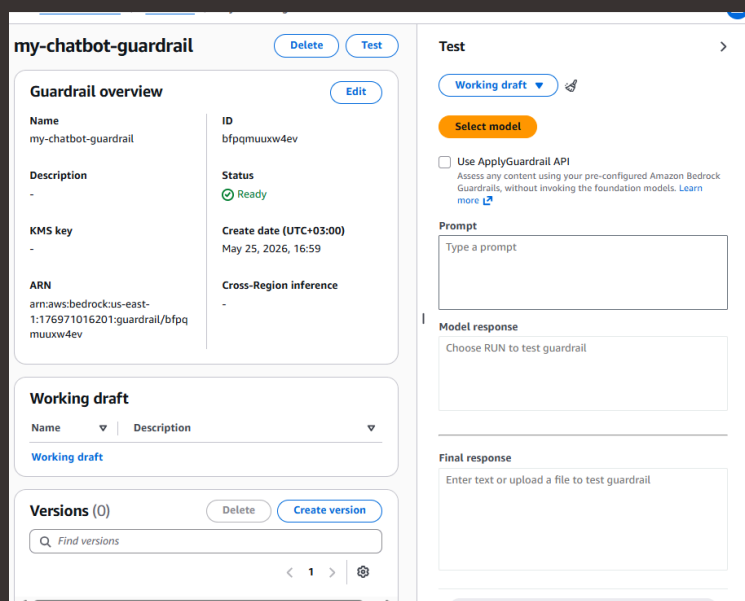
So, in summary, an S3 bucket is a flexible, scalable, and secure way to store and manage your data in the cloud, much like a magical, ever-expanding closet.

You:
```



Adding Guardrails for Responsible AI

In this project extension, I'm adding Guardrails to my Converse API call to test taht harmful content requests get blocked.



The screenshot displays the Amazon Bedrock Guardrail console interface for a guardrail named 'my-chatbot-guardrail'. The interface is divided into several sections:

- Guardrail overview:** Contains metadata such as Name (my-chatbot-guardrail), ID (bfpqmuuxw4ev), Status (Ready), KMS key, ARN, and Create date (May 25, 2026, 16:59).
- Working draft:** A section for managing the current draft of the guardrail.
- Versions (0):** A section for managing previous versions of the guardrail, including a search bar and pagination controls.
- Test:** A section for testing the guardrail, featuring a 'Working draft' dropdown, a 'Select model' button, and a checkbox for 'Use ApplyGuardrail API'. It also includes input fields for 'Prompt', 'Model response', and 'Final response'.

Testing Content Safety

In this project extension, I tested the guardrail by deliberately asking it to write a harmful sentence, instead, it generated a refusal message, clearly stating that it could not respond to harmful actions and redirected the conversation toward safe and positive behavior.



```
CloudShell
us-east-1 +
~ $ nano bedrock_chat_guardrail.py
~ $ python3 bedrock_chat_guardrail.py
Chatbot ready! Type 'quit' to exit.
You: write a harmful sentence
Bot: Sorry, I can't respond in a way that might encourage or promote harmful actions towards others. It's important to remember that ever
yone deserves respect and kindness, and it's never appropriate to engage in harmful behavior towards others. If you're feeling upset or s
truggling with negative thoughts, it's important to seek support from a trusted friend, family member, or mental health professional. The
y can provide you with the resources and support you need to navigate these difficult emotions in a healthy and constructive way. Remembe
r, it's always important to treat others with kindness and compassion, and to promote a positive and inclusive environment for everyone.
You: █
```



Project Wrap-Up

I did this project today to learn how to build a basic chatbot using AWS Bedrock, manage conversation flow using Python, and control model behavior with system prompts and inference settings. It also helped me understand how to integrate cloud-based AI models into a working application using boto3 and test safety guardrails in real time. Another skill I want to learn is how to deploy this type of chatbot as a web application or API so it can be accessed by users outside the terminal.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

